

SkillLink Freelance Marketplace

Backend Documentation

A Section of the DBMS Project Report

Submitted By:

Saim, Zarish, Maryam

BSIT, 5th Semester

Database Management Systems

Academic Year 2025–2026

Contents

1	Introduction to the Backend	5
1.1	What is the Backend?	5
1.2	The Role of the Backend in SkillLink	5
1.3	How the Backend Connects to the Database	6
1.4	The Three Layer Architecture	6
1.5	Technology Stack	6
1.6	Project File Structure	7
1.7	Route to Table Mapping	7
1.8	The app.js File	8
1.9	Summary	9
2	Database Connection	10
2.1	Overview	10
2.2	The .env File and Environment Variables	10
2.3	The db.js File	11
2.4	Why a Connection Pool is Used	11
2.5	Pool Configuration Parameters	12
2.6	The Warmup Query	12
2.7	The getConnection Function	12
2.8	The getConnection and close Pattern	13
2.9	How Oracle Queries Return Data	14
2.10	Bind Variables	14
2.11	Summary	14
3	Authentication and Middleware	16
3.1	Overview	16
3.2	The admins Table	16
3.3	User Registration	17
3.3.1	Email Existence Check	17
3.3.2	Insert into Users Table	17
3.3.3	Insert into Profile Table	18
3.3.4	Registration Database Flow	19
3.4	User Login	19
3.4.1	The Login Query	19
3.4.2	Why LEFT JOIN is Used Here	19
3.4.3	Password Check and Token Creation	20
3.5	Admin Login	20

3.5.1	The Admin Login Query	20
3.5.2	Comparison of User and Admin Login Queries	21
3.6	The verifyToken Middleware	21
3.6.1	How the Token is Extracted	22
3.6.2	How req.user is Used in Routes	22
3.6.3	The adminOnly Middleware Function	22
3.7	Authentication Flow Summary	23
3.8	Summary	23
4	User Facing Queries	24
4.1	Overview	24
4.2	Projects Queries	24
4.2.1	Get All Projects	24
4.2.2	Get Projects by Client	25
4.2.3	Get Single Project by ID	26
4.2.4	Post a New Project	26
4.2.5	Delete a Project	27
4.3	Bids Queries	27
4.3.1	Get Bids on a Project	27
4.3.2	Get Bids by Freelancer	28
4.3.3	Submit a Bid	28
4.4	Contracts Queries	29
4.4.1	Get Contracts for Logged In User	29
4.4.2	Get Single Contract by ID	30
4.5	Payments Queries	31
4.5.1	Get Payments for Logged In User	31
4.6	Reviews Queries	32
4.6.1	Get Reviews Received	32
4.6.2	Get Reviews Given	33
4.7	Disputes Queries	33
4.7.1	Get Disputes for Logged In User	33
4.7.2	Raise a Dispute	34
4.7.3	Resolve a Dispute	35
4.8	Notifications Queries	35
4.8.1	Get All Notifications for User	35
4.8.2	Get Unread Count	36
4.8.3	Mark Single Notification as Read	36
4.8.4	Mark All Notifications as Read	36
4.9	Summary of User Facing Queries	36
4.10	Summary	37
5	Transactions	38
5.1	Overview	38
5.2	How autoCommit Works in oracledb	38
5.3	Transaction 1: User Registration	39
5.3.1	Why a Transaction Is Needed	39
5.3.2	Step 1: Insert into Users	39
5.3.3	Step 2: Insert into Profile Table	40

5.3.4	Step 3: Commit	40
5.4	Transaction 2: Accept Bid	40
5.4.1	Step 1: Fetch the Bid	41
5.4.2	Step 2: Accept the Selected Bid	41
5.4.3	Step 3: Reject All Other Bids on the Same Project	41
5.4.4	Step 4: Update Project Status	41
5.4.5	Step 5: Create the Contract	42
5.4.6	Step 6: Insert Notification	42
5.4.7	Step 7: Commit	42
5.5	Transaction 3: Record Payment	43
5.5.1	Step 1: Verify the Contract	43
5.5.2	Step 2: Insert the Payment Row	43
5.5.3	Step 3: Mark Contract as Completed	44
5.5.4	Step 4: Mark Project as Completed	44
5.5.5	Step 5: Notify the Freelancer	44
5.5.6	Step 6: Commit	45
5.6	What Happens on Failure	45
5.7	Summary	46
6	Admin Queries	47
6.1	Overview	47
6.2	Dashboard Stats	47
6.2.1	The Nine Count Queries	47
6.2.2	How Promise.all Is Used	48
6.3	All Users	49
6.4	All Projects	50
6.5	All Bids	51
6.6	All Contracts	51
6.7	All Payments	52
6.8	All Reviews	53
6.9	All Skills	54
6.10	All Disputes	54
6.11	All Notifications	55
6.12	Summary of Admin Queries	56
6.13	Summary	56
7	API Reference	57
7.1	Overview	57
7.2	Authentication Endpoints	57
7.2.1	Register Request Body	58
7.2.2	Login Request Body	58
7.2.3	Login Response	59
7.3	Projects Endpoints	59
7.3.1	Post Project Request Body	60
7.4	Bids Endpoints	60
7.4.1	Submit Bid Request Body	61
7.5	Contracts Endpoints	61
7.6	Payments Endpoints	62

7.6.1	Record Payment Request Body	63
7.7	Reviews Endpoints	63
7.7.1	Submit Review Request Body	64
7.8	Disputes Endpoints	64
7.8.1	Raise Dispute Request Body	65
7.9	Notifications Endpoints	65
7.10	Admin Endpoints	66
7.11	HTTP Status Codes Used	68
7.12	Complete Endpoint Summary	68

Chapter 1

Introduction to the Backend

1.1 What is the Backend?

In any web application, the work is divided between two main parts. The frontend runs in the browser and is what the user sees and interacts with. The backend runs on a server and handles everything the user cannot see directly. This includes receiving requests from the frontend, running queries against the database, applying business rules, and sending the results back.

For SkillLink, the backend is built using Node.js with the Express framework. It acts as the bridge between the React frontend and the Oracle 21c XE database. The frontend never connects to the database directly. Every piece of data that appears on the screen first passes through the backend, which decides what query to run, who is allowed to see what, and what format to send the result in.

1.2 The Role of the Backend in SkillLink

SkillLink is a freelance marketplace where clients post projects and freelancers place bids on those projects. The backend is responsible for enforcing all the rules that make this process work correctly.

When a client registers, the backend inserts a new row into the users table and a corresponding row into the client profile table inside a single database transaction. When a freelancer logs in, the backend runs a SELECT query that joins the users table with the client profile and freelancer profile tables to determine their role. When a bid is accepted, the backend runs four separate SQL statements inside one transaction, updating the bids table, updating the projects table, and inserting a new row into the contracts table, all committed together so the database never ends up in an inconsistent state.

Every action a user takes on the frontend results in an HTTP request arriving at the backend. The backend verifies the identity of the user through a JWT token, runs the appropriate Oracle SQL query, and sends the result back as a JSON response.

1.3 How the Backend Connects to the Database

The backend connects to Oracle 21c XE using a Node.js package called `oracledb`. This package provides a connection pool that keeps several database connections open and ready so that incoming requests do not have to wait for a new connection to be established each time.

The connection details are stored in a file called `db.js` inside the `config` folder. This file creates the pool when the server starts and exports a `getConnection` function that every route file uses to borrow a connection from the pool, run its query, and return the connection when done.

The database credentials are never hardcoded in the source files. They are stored in a separate file called `.env` which is loaded at startup using the `dotenv` package. This file contains the Oracle username, password, and connect string, along with the JWT secret key used for signing authentication tokens.

1.4 The Three Layer Architecture

SkillLink uses a three layer architecture where each layer has a clearly defined responsibility. The frontend handles the user interface. The backend handles business logic and database access. The database stores all persistent data.

Layer	Technology	Responsibility
Frontend	React.js	User interface, HTTP requests
Backend	Node.js with Express	Business logic, SQL queries
Database	Oracle 21c XE	Persistent storage, 15 tables

Table 1.1: Three Layer Architecture of SkillLink

The backend sits in the middle of this stack. It is the only layer that talks to both the frontend and the database. The frontend never runs SQL. The database never processes business rules. Everything meets in the backend.

1.5 Technology Stack

The backend uses a small set of well chosen packages, each with a specific job.

Package	Purpose
<code>express</code>	HTTP server and routing framework
<code>oracledb</code>	Oracle database driver and connection pool
<code>dotenv</code>	Loads environment variables from <code>.env</code> file
<code>jsonwebtoken</code>	Creates and verifies JWT authentication tokens
<code>cors</code>	Allows the React frontend to send requests to the backend
<code>bcrypt</code>	Password hashing (available, used optionally)

Table 1.2: Backend Package Dependencies

1.6 Project File Structure

All backend code lives inside a single folder. The structure is flat and easy to follow. Each route file handles one area of the application and maps directly to one or more tables in the Oracle database.

Listing 1.1: Backend Folder Structure

```

1 backend/
2   app.js           <- main server file
3   .env            <- environment variables
4   config/
5     db.js         <- Oracle connection pool
6   middleware/
7     auth.js       <- JWT verification
8   routes/
9     auth.js       <- register and login
10    projects.js    <- project CRUD
11    bids.js        <- bid submission and acceptance
12    contracts.js   <- contract retrieval
13    payments.js    <- payment recording
14    reviews.js     <- review submission
15    disputes.js    <- dispute management
16    notifications.js <- notification management
17    adminAuth.js   <- admin login
18    admin.js       <- all admin data views

```

1.7 Route to Table Mapping

Each route file in the backend is responsible for a specific set of Oracle tables. The table below shows this relationship clearly.

Route File	Oracle Tables Involved
auth.js	users, client_profile, freelancer_profile
projects.js	projects, users, project_category, categories, bids
bids.js	bids, projects, users, freelancer_profile, contracts
contracts.js	contracts, bids, projects, users
payments.js	payments, contracts, bids, projects, users
reviews.js	reviews, contracts, bids, projects, users, freelancer_profile
disputes.js	disputes, contracts, bids, projects, users
notifications.js	notifications
adminAuth.js	admins
admin.js	All 15 tables

Table 1.3: Route Files and their Oracle Tables

1.8 The app.js File

The app.js file is the entry point of the entire backend. It creates the Express application, registers the CORS policy, registers all route files under their URL prefixes, and starts the server after the Oracle connection pool is ready.

Listing 1.2: app.js - Main Server Entry Point

```
1  const express = require('express');
2  const cors = require('cors');
3  require('dotenv').config();
4  const { connectDB } = require('./config/db');
5
6  const app = express();
7
8  app.use(cors({
9    origin: 'http://localhost:3000',
10   methods: ['GET', 'POST', 'PUT', 'DELETE'],
11   allowedHeaders: ['Content-Type', 'Authorization'],
12 }));
13
14 app.use(express.json());
15
16 app.use('/api/auth',      require('./routes/auth'));
17 app.use('/api/projects',  require('./routes/projects'));
18 app.use('/api/bids',      require('./routes/bids'));
19 app.use('/api/contracts', require('./routes/contracts'));
20 app.use('/api/payments',  require('./routes/payments'));
21 app.use('/api/reviews',   require('./routes/reviews'));
22 app.use('/api/notifications',
23       require('./routes/notifications'));
24 app.use('/api/disputes',   require('./routes/disputes'));
25 app.use('/api/admin/auth', require('./routes/adminAuth'));
26 app.use('/api/admin',     require('./routes/admin'));
27
28 const PORT = process.env.PORT || 5000;
29
30 connectDB().then(() => {
31   app.listen(PORT, () => {
32     console.log('Server running on port ${PORT}');
33   });
34 });
```

The order of route registration matters for the admin routes. The `/api/admin/auth` route must be registered before `/api/admin` so that login requests are not accidentally caught by the broader admin route handler.

The server only starts listening for requests after the `connectDB` function resolves successfully. If the Oracle database is not running or the credentials are wrong, the process exits before any requests can be served. This prevents the backend from accepting requests when it has no database to talk to.

1.9 Summary

The SkillLink backend is a Node.js application that sits between the React frontend and the Oracle 21c XE database. It receives HTTP requests, verifies the identity of the user through JWT tokens, runs the appropriate SQL query against the database, and sends the result back as JSON. It is organized into ten route files each responsible for a specific set of Oracle tables. The following chapters go through each part of the backend in detail, with particular focus on the SQL queries that drive every operation in the system.

Chapter 2

Database Connection

2.1 Overview

Every route in the SkillLink backend needs to communicate with the Oracle 21c XE database. This communication is managed entirely by a single file called `db.js` inside the `config` folder. This file is responsible for creating the connection pool when the server starts, warming it up with a test query, and exporting a function that every route file calls when it needs to run a query.

This chapter explains how the connection pool works, why it is used instead of creating a new connection for every request, and how the `getConnection` and `close` pattern is applied consistently across all route files.

2.2 The `.env` File and Environment Variables

Before the backend can connect to Oracle, it needs to know the database credentials. These are never written directly into the source code. Instead they are stored in a separate file called `.env` at the root of the backend folder. This file is loaded at startup by the `dotenv` package, which makes each variable available through `process.env`.

Listing 2.1: The `.env` File

```
1 PORT=5000
2 DB_USER=freelance
3 DB_PASSWORD=1234
4 DB_CONNECT_STRING=localhost:1521/XE
5 JWT_SECRET=skilllink_secret_key_2026
```

The `DB_CONNECT_STRING` tells `oracledb` where Oracle is running. The value `localhost:1521/XE` means the database is on the same machine as the backend, listening on port 1521, with a service name of XE. This is the standard configuration for Oracle 21c XE installed locally.

The `JWT_SECRET` is not related to the database connection but is stored here alongside the database credentials so all sensitive configuration is in one place.

2.3 The db.js File

The complete db.js file is shown below. The sections that follow explain each part in detail.

Listing 2.2: config/db.js - Connection Pool Setup

```
1  const oracledb = require('oracledb');
2  require('dotenv').config();
3
4  let pool;
5
6  async function connectDB() {
7    try {
8      pool = await oracledb.createPool({
9        user:          process.env.DB_USER,
10       password:       process.env.DB_PASSWORD,
11       connectString: process.env.DB_CONNECT_STRING,
12       poolMin:        2,
13       poolMax:        10,
14       poolIncrement: 1,
15     });
16
17     const conn = await pool.getConnection();
18     await conn.execute('SELECT 1 FROM DUAL');
19     await conn.close();
20
21     console.log('Oracle DB connected successfully');
22   } catch (err) {
23     console.error('DB Connection Failed:', err);
24     process.exit(1);
25   }
26 }
27
28 async function getConnection() {
29   return await pool.getConnection();
30 }
31
32 module.exports = { connectDB, getConnection };
```

2.4 Why a Connection Pool is Used

A connection pool is a set of database connections that are created once when the server starts and then reused across multiple requests. Without a pool, the backend would have to create a brand new Oracle connection every time a request arrived. Creating a connection involves a network handshake with Oracle, authentication, and session setup. This takes time and would make the application slow under any load.

With a pool, connections are already open and waiting. When a request arrives, the route borrows one connection from the pool, runs its query, and returns the connection.

The next request can reuse that same connection immediately.

2.5 Pool Configuration Parameters

The `createPool` call passes three sizing parameters that control how the pool behaves.

Parameter	Value	Meaning
<code>poolMin</code>	2	Keep at least 2 connections open at all times
<code>poolMax</code>	10	Never open more than 10 connections simultaneously
<code>poolIncrement</code>	1	When more connections are needed, add 1 at a time

Table 2.1: Oracle Connection Pool Parameters

The `poolMin` value of 2 means that even when no requests are being processed, two connections remain open and ready. This eliminates the delay on the first request after a quiet period.

The `poolMax` value of 10 acts as a safety limit. Oracle has a maximum number of sessions it can handle. Setting `poolMax` prevents the backend from overwhelming the database by opening too many connections under heavy load.

The `poolIncrement` value of 1 means the pool grows gradually rather than opening several connections at once when demand increases suddenly.

2.6 The Warmup Query

After the pool is created, the `connectDB` function immediately borrows one connection and runs a simple query against Oracle's built in `DUAL` table.

Listing 2.3: Warmup Query Against `DUAL`

```
1 SELECT 1 FROM DUAL
```

The `DUAL` table is a special single row single column table that exists in every Oracle database. It is used here purely to verify that the connection works. If this query succeeds, the backend knows that Oracle is running, the credentials are correct, and the network path is open. The connection is then closed back to the pool and the success message is printed.

If the warmup query fails for any reason, the catch block runs and the process exits immediately with `process.exit(1)`. This is intentional. There is no point starting the HTTP server if there is no database to talk to. The error message printed to the console tells the developer exactly what went wrong so they can fix it before trying again.

2.7 The `getConnection` Function

The second function exported from `db.js` is `getConnection`. This is the function that every route file calls when it needs to run a query.

Listing 2.4: The getConnection Function

```
1 async function getConnection() {  
2   return await pool.getConnection();  
3 }
```

This function simply borrows one connection from the pool and returns it to the caller. The caller is responsible for closing the connection when it is done. Closing a pooled connection does not actually close the underlying Oracle session. It returns the connection to the pool so it can be reused by the next request.

2.8 The getConnection and close Pattern

Every route file in SkillLink follows the same three step pattern when running a query. The first step is to get a connection from the pool. The second step is to execute the query. The third step is to close the connection back to the pool.

Listing 2.5: Standard Query Pattern Used in All Routes

```
1 const connection = await getConnection();  
2  
3 const result = await connection.execute(  
4   'SELECT column FROM table WHERE id = :id',  
5   { id: someValue }  
6 );  
7  
8 await connection.close();  
9  
10 res.json(result.rows);
```

The close call must happen in every code path, including error paths. If a validation check fails midway through a route and the function returns early, the connection must still be closed before returning. This is why many routes call `connection.close()` inside the early return block before sending the error response.

Listing 2.6: Closing Connection Before Early Return

```
1 const result = await connection.execute(  
2   'SELECT status FROM projects WHERE project_id = :id',  
3   { id: project_id }  
4 );  
5  
6 if (result.rows[0][0] !== 'open') {  
7   await connection.close();  
8   return res.status(400).json({  
9     error: 'Project is not open for bids'  
10  });  
11 }
```

If the connection is not closed and is simply left open, it is never returned to the pool. After enough such leaks, the pool runs out of available connections and all new requests hang indefinitely waiting for a connection that never becomes available.

2.9 How Oracle Queries Return Data

When `oracledb` executes a `SELECT` query, the result object contains a `rows` array. Each element of this array is itself an array representing one row from the result set. The columns appear in the same order as they are listed in the `SELECT` statement.

Listing 2.7: Accessing Query Results

```
1 const result = await connection.execute(  
2   'SELECT user_id, name, email FROM users WHERE user_id =  
3     :id',  
4   { id: 1 }  
5 );  
6 // result.rows[0][0] is user_id  
7 // result.rows[0][1] is name  
8 // result.rows[0][2] is email
```

This is why the frontend accesses project data using array indices like `p[0]` for the project ID and `p[1]` for the title. The column order in the `SELECT` statement determines the index used on the frontend.

2.10 Bind Variables

All queries in SkillLink use bind variables rather than string concatenation. A bind variable is a placeholder in the SQL statement that is filled in by `oracledb` separately from the query text. They are written with a colon prefix in Oracle SQL.

Listing 2.8: Bind Variable Syntax in Oracle SQL

```
1 SELECT user_id, name FROM users WHERE email = :email
```

Listing 2.9: Passing Bind Variables in `oracledb`

```
1 const result = await connection.execute(  
2   'SELECT user_id, name FROM users WHERE email = :email',  
3   { email: req.body.email }  
4 );
```

Bind variables protect against SQL injection attacks. If a user submitted a malicious string as their email address, string concatenation would allow it to modify the query itself. With bind variables, the user input is always treated as a value and never as part of the query structure. Oracle handles the separation internally.

Bind variables also improve performance because Oracle can cache the execution plan for a query and reuse it when the same query runs again with different values.

2.11 Summary

The `db.js` file is the foundation of the entire backend. It creates a pool of Oracle connections when the server starts, verifies the connection with a warmup query against

DUAL, and exports the `getConnection` function that all route files use. The pool keeps connections ready so requests are served quickly. Every route follows the same pattern of getting a connection, executing a query, and closing the connection. Bind variables are used in every query to protect against SQL injection and improve performance. The following chapter covers the authentication system, which is the first set of queries that runs when a user interacts with the application.

Chapter 3

Authentication and Middleware

3.1 Overview

Authentication is the process of verifying who a user is before allowing them to access the system. In SkillLink, authentication is handled by two route files and one middleware file. The `auth.js` route handles user registration and login. The `adminAuth.js` route handles admin login. The `middleware/auth.js` file verifies the JWT token on every protected request.

This chapter explains each of these three files in detail, with full SQL queries shown for every database operation involved in the authentication process.

3.2 The admins Table

Before covering the route files, it is important to understand the database tables involved in authentication. Regular users are stored in the `users` table. Their role is determined by which profile table has a matching record for their user ID. Admins are stored in a completely separate table called `admins`.

Listing 3.1: The admins Table Structure

```
1 CREATE TABLE admins (  
2   admin_id    NUMBER PRIMARY KEY,  
3   name        VARCHAR2(100) NOT NULL,  
4   email       VARCHAR2(100) UNIQUE NOT NULL,  
5   password    VARCHAR2(100) NOT NULL,  
6   created_at  DATE DEFAULT SYSDATE  
7 );
```

Keeping admins in a separate table means that even if a regular user somehow obtained admin credentials, they would still be rejected because the admin login route queries a completely different table. The two authentication paths never share a query.

3.3 User Registration

When a new user submits the registration form on the frontend, a POST request arrives at the backend with the user's name, email, password, phone, and role. The role is either client or freelancer.

3.3.1 Email Existence Check

Before inserting anything, the backend runs a SELECT query to check whether the email address is already registered.

Listing 3.2: Query 1: Check for Existing Email

```
1 SELECT user_id
2 FROM users
3 WHERE email = :email
```

If this query returns any rows, the backend stops immediately and returns a 409 Conflict response. The insert queries never run. This prevents duplicate accounts and protects the UNIQUE constraint on the email column of the users table.

3.3.2 Insert into Users Table

If the email is new, the backend inserts a row into the users table. The primary key is generated automatically using the seq_users sequence, which was created alongside the tables to provide auto incrementing IDs.

Listing 3.3: Query 2: Insert New User

```
1 INSERT INTO users
2 VALUES (
3     seq_users.nextval,
4     :name,
5     :email,
6     :password,
7     :phone,
8     sysdate
9 )
10 RETURNING user_id INTO :user_id
```

The RETURNING clause is an Oracle specific feature that sends the newly generated user_id back to the backend immediately after the insert. This is necessary because the next insert into the profile table needs that user_id as its primary key. Without RETURNING, the backend would need to run a second SELECT query to find the ID that was just created.

In oracledb, the RETURNING clause requires a special bind variable configuration.

Listing 3.4: RETURNING Clause in oracledb

```
1 const result = await connection.execute(
2     'INSERT INTO users
```

```
3      VALUES (seq_users.nextval, :name, :email,
4               :password, :phone, sysdate)
5      RETURNING user_id INTO :user_id',
6      {
7          name,
8          email,
9          password,
10         phone: phone || null,
11         user_id: {
12             dir: oracledb.BIND_OUT,
13             type: oracledb.NUMBER
14         }
15     },
16     { autoCommit: false }
17 );
18
19 const userId = result.outBinds.user_id[0];
```

The autoCommit is set to false because this insert is part of a transaction. The commit only happens after the profile insert also succeeds.

3.3.3 Insert into Profile Table

After the user row is created, the backend inserts into either `client_profile` or `freelancer_profile` depending on the role value.

Listing 3.5: Query 3a: Insert into `client_profile`

```
1 INSERT INTO client_profile
2 VALUES (:id, :company_name, :location)
```

Listing 3.6: Query 3b: Insert into `freelancer_profile`

```
1 INSERT INTO freelancer_profile
2 VALUES (
3     :id,
4     :bio,
5     :hourly_rate,
6     0,
7     :experience_level,
8     'yes'
9 )
```

The `client_id` and `freelancer_id` columns in these tables are not separate sequences. They are the same `user_id` that was just returned from the users insert. This is what creates the one to one relationship between a user and their profile.

After both inserts succeed, the transaction is committed.

Listing 3.7: Committing the Registration Transaction

```
1 await connection.commit();
2 await connection.close();
```

If either insert fails for any reason, neither insert is committed to the database. The user does not end up with a row in users but no matching profile row.

3.3.4 Registration Database Flow

Step	Operation	Table
1	SELECT to check email	users
2	INSERT new user with RETURNING	users
3	INSERT profile based on role	client_profile or freelancer_profile
4	COMMIT transaction	both tables

Table 3.1: Database Operations During User Registration

3.4 User Login

The login query is the most important SELECT query in the entire backend because it determines who the user is and what role they have. This role is then encoded into the JWT token and carried through every subsequent request.

3.4.1 The Login Query

Listing 3.8: Query 4: User Login with Role Detection

```

1 SELECT u.user_id,
2       u.name,
3       u.email,
4       u.password,
5       CASE
6         WHEN c.client_id IS NOT NULL THEN 'client'
7         WHEN f.freelancer_id IS NOT NULL THEN 'freelancer'
8       END as role
9 FROM users u
10 LEFT JOIN client_profile c ON u.user_id = c.client_id
11 LEFT JOIN freelancer_profile f ON u.user_id = f.freelancer_id
12 WHERE u.email = :email

```

This query joins three tables in a single SELECT. The LEFT JOIN with client_profile and freelancer_profile means the query returns a row even if neither profile table has a match, which would result in a null role. The CASE WHEN expression checks which profile table has a matching record and sets the role accordingly.

3.4.2 Why LEFT JOIN is Used Here

A regular INNER JOIN would only return rows where both sides of the join have a match. Since every user has exactly one profile, an inner join would work in most cases. However, a LEFT JOIN is safer because it ensures the query always returns the user row even if the profile data is somehow missing. This prevents a situation where a valid user cannot log in because of a missing profile row.

3.4.3 Password Check and Token Creation

After the query returns, the backend compares the submitted password with the stored password. If they match, a JWT token is created and sent back to the frontend.

Listing 3.9: Password Check and JWT Token Creation

```
1 const user = result.rows[0];
2
3 if (password !== user[3])
4   return res.status(401).json({
5     error: 'Invalid email or password'
6   });
7
8 const token = jwt.sign(
9   {
10     user_id: user[0],
11     email:   user[2],
12     role:    user[4]
13   },
14   process.env.JWT_SECRET,
15   { expiresIn: '7d' }
16 );
```

The token payload contains three claims: the `user_id`, the `email`, and the `role`. The `role` value came directly from the `CASE WHEN` expression in the Oracle query. This is the value that controls everything the user can see and do for the next seven days until the token expires.

3.5 Admin Login

The admin login follows the same pattern as user login but queries the `admins` table instead of the `users` and `profile` tables. The route is registered at a completely separate URL prefix and the token it issues carries the role of `admin` rather than `client` or `freelancer`.

3.5.1 The Admin Login Query

Listing 3.10: Query 5: Admin Login

```
1 SELECT admin_id, name, email, password
2 FROM admins
3 WHERE email = :email
```

This query is simpler than the user login query because there are no profile tables to join and no `CASE WHEN` needed. The `admins` table stores everything in one place. The `role` is not determined by the query result. It is hardcoded as `admin` in the JWT token because anyone who successfully authenticates against the `admins` table is by definition an admin.

Listing 3.11: Admin Token Creation

```

1 const token = jwt.sign(
2   {
3     admin_id: admin[0],
4     email:    admin[2],
5     role:     'admin'
6   },
7   process.env.JWT_SECRET,
8   { expiresIn: '7d' }
9 );

```

3.5.2 Comparison of User and Admin Login Queries

Aspect	User Login	Admin Login
Primary table	users	admins
Tables joined	client_profile, freelancer_profile	none
Role detection	CASE WHEN from JOIN	hardcoded as admin
Token claim	user_id	admin_id
Route	/api/auth/login	/api/admin/auth/login

Table 3.2: Comparison of User and Admin Login Queries

3.6 The verifyToken Middleware

After login, every protected request from the frontend includes the JWT token in the Authorization header. The verifyToken middleware reads this header, verifies the token, and attaches the decoded payload to the request object so that route handlers can access the user's identity without running another database query.

Listing 3.12: middleware/auth.js - JWT Verification

```

1 const jwt = require('jsonwebtoken');
2 require('dotenv').config();
3
4 module.exports = function verifyToken(req, res, next) {
5   const authHeader = req.headers['authorization'];
6   const token = authHeader && authHeader.split(' ')[1];
7
8   if (!token)
9     return res.status(401).json({
10       error: 'Access denied. No token provided.'
11     });
12
13   try {
14     const decoded = jwt.verify(token, process.env.JWT_SECRET);
15     req.user = decoded;
16     next();
17   } catch (err) {
18     return res.status(403).json({

```

```
19         error: 'Invalid or expired token.'
20     });
21 }
22 };
```

3.6.1 How the Token is Extracted

The Authorization header follows the Bearer scheme. The frontend sends the header in this format.

Listing 3.13: Authorization Header Format

```
1 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

The middleware splits this string on the space character and takes the second part, which is the token itself. If the header is missing or malformed, the token variable is undefined and the middleware returns a 401 Unauthorized response immediately.

3.6.2 How req.user is Used in Routes

After the middleware verifies the token, it attaches the decoded payload to req.user. This object is then available in every route handler that comes after the middleware in the chain.

Listing 3.14: Accessing req.user in a Route Handler

```
1 router.get('/my', verifyToken, async (req, res) => {
2     const user_id = req.user.user_id;
3     const role     = req.user.role;
4
5     // user_id and role are now available without
6     // running any database query
7 });
```

This is why no route ever needs to look up who the user is from the database. The identity is already verified and available in req.user. The user_id is used in WHERE clauses to filter data to the logged in user. The role is used to decide which query to run or whether to allow an operation at all.

3.6.3 The adminOnly Middleware Function

The admin routes use an additional inline check on top of verifyToken. This function is defined inside admin.js and runs after verifyToken has already verified the token.

Listing 3.15: adminOnly Middleware in admin.js

```
1 function adminOnly(req, res, next) {
2     if (req.user.role !== 'admin')
3         return res.status(403).json({
4             error: 'Admin access only'
5         });
6     next();
7 }
```

7 | }

Even if someone has a valid JWT token as a client or freelancer, they cannot access any admin endpoint because this check rejects any token whose role is not exactly admin. The role value was set at login time by the Oracle query and cannot be changed without a new login.

3.7 Authentication Flow Summary

Step	Operation	Where it Happens
1	User submits registration form	Frontend
2	Backend checks email, inserts 3 rows	auth.js + Oracle
3	User submits login form	Frontend
4	Backend queries 3 tables, creates token	auth.js + Oracle
5	Frontend stores token in local storage	Frontend
6	Every request includes token in header	Frontend
7	verifyToken extracts and verifies token	middleware/auth.js
8	req.user is available in route handler	All protected routes

Table 3.3: End to End Authentication Flow

3.8 Summary

The authentication system in SkillLink is built on five Oracle queries. The email check prevents duplicate registrations. The two insert queries create the user and their profile inside a transaction. The login query uses a three table LEFT JOIN with a CASE WHEN expression to determine the user role in a single database call. The admin login queries a separate table and issues a token with a hardcoded admin role. The verifyToken middleware verifies every subsequent request without touching the database, using only the signed token issued at login time. The following chapter covers the user facing queries that power every page of the application.

Chapter 4

User Facing Queries

4.1 Overview

This chapter covers every SQL query used in the six user facing route files: projects, bids, contracts, payments, reviews, disputes, and notifications. For each query the full SQL is shown, the tables involved are identified, and the purpose of every JOIN and WHERE clause is explained.

These queries are the core of the SkillLink backend. Every piece of data that appears on a user's screen comes from one of the queries in this chapter.

4.2 Projects Queries

The projects route file contains five operations. It is the most used route in the application because projects are the central entity that connects clients, freelancers, bids, and contracts.

4.2.1 Get All Projects

This query is called when a freelancer opens the Browse Projects page. It returns all projects on the platform regardless of who posted them.

Listing 4.1: Query 1: Select All Projects

```
1 SELECT p.project_id ,
2       p.title ,
3       p.description ,
4       p.budget ,
5       p.deadline ,
6       p.status ,
7       u.name      AS client_name ,
8       u.user_id AS client_id ,
9       c.name      AS category ,
10      (SELECT COUNT(*)
11       FROM bids b
12       WHERE b.project_id = p.project_id) AS bid_count
```

```

13 FROM projects p
14 JOIN users u
15     ON p.client_id = u.user_id
16 LEFT JOIN project_category pc
17     ON p.project_id = pc.project_id
18 LEFT JOIN categories c
19     ON pc.category_id = c.category_id
20 ORDER BY p.project_id DESC

```

The JOIN with users brings in the client name alongside each project. The LEFT JOIN with project_category and categories is used because not every project has a category assigned. A LEFT JOIN ensures projects without a category still appear in the results rather than being excluded by an inner join. The correlated subquery counts the total number of bids on each project by scanning the bids table for each project row.

Column	Source Table
project_id, title, description, budget, deadline, status	projects
client_name, client_id	users
category	categories
bid_count	bids (subquery)

Table 4.1: Column Sources for Get All Projects Query

4.2.2 Get Projects by Client

This query is called when a client opens the My Projects page. It returns only the projects posted by the logged in client, identified by the user_id extracted from the JWT token.

Listing 4.2: Query 2: Select Projects by Client ID

```

1  SELECT p.project_id,
2         p.title,
3         p.description,
4         p.budget,
5         p.deadline,
6         p.status,
7         c.name AS category,
8         (SELECT COUNT(*)
9          FROM bids b
10         WHERE b.project_id = p.project_id) AS bid_count,
11        (SELECT COUNT(*)
12         FROM bids b
13         WHERE b.project_id = p.project_id
14         AND   b.status = 'pending') AS pending_bids
15 FROM projects p
16 LEFT JOIN project_category pc
17     ON p.project_id = pc.project_id
18 LEFT JOIN categories c
19     ON pc.category_id = c.category_id
20 WHERE p.client_id = :client_id

```

```
21 ORDER BY p.project_id DESC
```

This query has two subqueries instead of one. The first counts all bids. The second counts only pending bids. The client uses this to see at a glance how many bids are still waiting for a decision on each of their projects. The WHERE clause filters results to the logged in client using the bind variable `client_id`.

4.2.3 Get Single Project by ID

This query fetches a single project row by its primary key. It is used when the backend needs to verify project details before processing a bid.

Listing 4.3: Query 3: Select Single Project

```
1 SELECT p.project_id ,
2       p.title ,
3       p.description ,
4       p.budget ,
5       p.deadline ,
6       p.status ,
7       u.name      AS client_name ,
8       u.user_id AS client_id ,
9       c.name      AS category
10 FROM projects p
11 JOIN users u
12   ON p.client_id = u.user_id
13 LEFT JOIN project_category pc
14   ON p.project_id = pc.project_id
15 LEFT JOIN categories c
16   ON pc.category_id = c.category_id
17 WHERE p.project_id = :id
```

4.2.4 Post a New Project

When a client posts a new project, the backend runs an INSERT using the `seq_projects` sequence for the primary key. The `client_id` comes from `req.user.user_id` and is never taken from the request body. The status is hardcoded to open because all new projects begin in the open state.

Listing 4.4: Query 4: Insert New Project

```
1 INSERT INTO projects
2 VALUES (
3     seq_projects.nextval ,
4     :client_id ,
5     :title ,
6     :description ,
7     :budget ,
8     TO_DATE(:deadline , 'YYYY-MM-DD') ,
9     'open'
10 )
```

The `TO_DATE` function converts the deadline string sent from the frontend into an Oracle `DATE` type. The format mask `YYYY-MM-DD` matches the ISO date format that the HTML date input on the frontend produces.

4.2.5 Delete a Project

Before deleting a project, the backend checks that no bids have been placed on it. A project that has bids cannot be deleted because the bids reference it through a foreign key constraint.

Listing 4.5: Query 5a: Count Bids Before Delete

```
1 SELECT COUNT(*)
2 FROM bids
3 WHERE project_id = :id
```

If the count is greater than zero the delete is rejected. If no bids exist the delete proceeds.

Listing 4.6: Query 5b: Delete Project

```
1 DELETE FROM projects
2 WHERE project_id = :id
3 AND client_id = :client_id
```

The `AND client_id = :client_id` condition ensures a client can only delete their own projects. Even if someone obtained a valid token and guessed a project ID, they could not delete a project that belongs to another client.

4.3 Bids Queries

The bids route file contains four operations. The most important is the accept bid operation which triggers a multi table transaction covered in detail in Chapter 5.

4.3.1 Get Bids on a Project

This query is called when a client selects a project on the Bids Received page. It returns all bids placed on that project along with detailed information about each freelancer.

Listing 4.7: Query 6: Select Bids for a Project

```
1 SELECT b.bid_id ,
2        b.amount ,
3        b.delivery_time ,
4        b.status ,
5        b.submitted_at ,
6        u.name    AS freelancer_name ,
7        u.email    AS freelancer_email ,
8        u.user_id AS freelancer_id ,
9        f.rating ,
10       f.experience_level ,
11       f.bio
12 FROM bids b
```

```
13 JOIN users u
14     ON b.freelancer_id = u.user_id
15 JOIN freelancer_profile f
16     ON u.user_id = f.freelancer_id
17 WHERE b.project_id = :project_id
18 ORDER BY b.amount ASC
```

The JOIN with users brings in the freelancer name and email. The JOIN with freelancer_profile brings in the rating, experience level, and bio so the client can evaluate each bidder without leaving the page. The results are ordered by amount ascending so the lowest bid appears first, which is the most useful ordering for a client comparing bids.

4.3.2 Get Bids by Freelancer

This query is called when a freelancer opens their My Bids page. It returns all bids that freelancer has submitted across all projects.

Listing 4.8: Query 7: Select Bids by Freelancer

```
1 SELECT b.bid_id,
2         b.amount,
3         b.delivery_time,
4         b.status,
5         b.submitted_at,
6         p.title      AS project_title,
7         p.budget,
8         p.status      AS project_status,
9         p.project_id,
10        u.name        AS client_name
11 FROM bids b
12 JOIN projects p
13     ON b.project_id = p.project_id
14 JOIN users u
15     ON p.client_id = u.user_id
16 WHERE b.freelancer_id = :freelancer_id
17 ORDER BY b.submitted_at DESC
```

The JOIN with projects gives the freelancer the project title and budget alongside each bid. The JOIN with users gives the client name. The results are ordered by submission date descending so the most recent bid appears first.

4.3.3 Submit a Bid

Before inserting a new bid, the backend runs two validation queries. The first checks that the project is still open. The second checks that the freelancer has not already bid on this project.

Listing 4.9: Query 8a: Check Project Status Before Bid

```
1 SELECT status, client_id
2 FROM projects
3 WHERE project_id = :id
```

Listing 4.10: Query 8b: Check for Duplicate Bid

```
1 SELECT bid_id
2 FROM bids
3 WHERE project_id = :pid
4 AND freelancer_id = :fid
```

If both checks pass, the bid is inserted with a status of pending.

Listing 4.11: Query 8c: Insert New Bid

```
1 INSERT INTO bids
2 VALUES (
3     seq_bids.nextval,
4     :project_id,
5     :freelancer_id,
6     :amount,
7     :delivery_time,
8     'pending',
9     sysdate
10 )
```

The `freelancer_id` is taken from `req.user.user_id` and never from the request body. This ensures a freelancer can only bid under their own identity.

4.4 Contracts Queries

The contracts route file contains two `SELECT` queries. Both join multiple tables to return complete contract information. The same endpoint serves both clients and freelancers but runs a different query depending on the role in the JWT token.

4.4.1 Get Contracts for Logged In User

The backend checks `req.user.role` and builds the appropriate query. The difference between the two versions is which side of the contract the `WHERE` clause filters on.

Listing 4.12: Query 9a: Contracts for Client

```
1 SELECT c.contract_id,
2        c.amount,
3        c.start_date,
4        c.end_date,
5        c.status,
6        p.title AS project_title,
7        p.project_id,
8        u.name AS freelancer_name,
9        u.email AS freelancer_email,
10       u.user_id AS freelancer_id
11 FROM contracts c
12 JOIN bids b
13     ON c.bid_id = b.bid_id
14 JOIN projects p
```

```

15     ON b.project_id = p.project_id
16 JOIN users u
17     ON b.freelancer_id = u.user_id
18 WHERE p.client_id = :user_id
19 ORDER BY c.start_date DESC

```

Listing 4.13: Query 9b: Contracts for Freelancer

```

1 SELECT c.contract_id,
2         c.amount,
3         c.start_date,
4         c.end_date,
5         c.status,
6         p.title      AS project_title,
7         p.project_id,
8         u.name       AS client_name,
9         u.email      AS client_email,
10        u.user_id    AS client_id
11 FROM contracts c
12 JOIN bids b
13     ON c.bid_id = b.bid_id
14 JOIN projects p
15     ON b.project_id = p.project_id
16 JOIN users u
17     ON p.client_id = u.user_id
18 WHERE b.freelancer_id = :user_id
19 ORDER BY c.start_date DESC

```

Both queries follow the same JOIN chain: contracts joins to bids, bids joins to projects, and projects or bids joins to users. The difference is which users JOIN is used and which WHERE clause is applied. For a client the filter is on p.client_id. For a freelancer the filter is on b.freelancer_id.

Aspect	Client Query	Freelancer Query
users JOIN	ON b.freelancer_id = u.user_id	ON p.client_id = u.user_id
WHERE clause	p.client_id = :user_id	b.freelancer_id = :user_id
Other party shown	Freelancer name	Client name

Table 4.2: Difference Between Client and Freelancer Contract Queries

4.4.2 Get Single Contract by ID

This query fetches one contract by its primary key. It joins five tables and uses two aliases for the users table to get both the client name and the freelancer name in a single query.

Listing 4.14: Query 10: Select Single Contract

```

1 SELECT c.contract_id,
2         c.amount,
3         c.start_date,

```

```
4      c.end_date ,
5      c.status ,
6      p.title     AS project_title ,
7      p.project_id ,
8      uc.name     AS client_name ,
9      uc.user_id  AS client_id ,
10     uf.name     AS freelancer_name ,
11     uf.user_id  AS freelancer_id
12 FROM contracts c
13 JOIN bids b
14     ON c.bid_id = b.bid_id
15 JOIN projects p
16     ON b.project_id = p.project_id
17 JOIN users uc
18     ON p.client_id = uc.user_id
19 JOIN users uf
20     ON b.freelancer_id = uf.user_id
21 WHERE c.contract_id = :id
```

The users table is joined twice using two different aliases. The alias uc represents the client and is joined through the projects table. The alias uf represents the freelancer and is joined through the bids table. This pattern of aliasing the same table twice to get data from both sides of a relationship is used in several queries throughout the admin routes as well.

4.5 Payments Queries

The payments route file contains two operations. The GET query fetches payment history filtered by role. The POST operation is a transaction covered in Chapter 5.

4.5.1 Get Payments for Logged In User

Like the contracts query, the payments query branches based on role. Both versions follow the same five table JOIN chain but filter on different columns.

Listing 4.15: Query 11a: Payments for Client

```
1 SELECT pay.payment_id ,
2       pay.amount ,
3       pay.pay_date ,
4       pay.method ,
5       pay.status ,
6       p.title     AS project_title ,
7       uf.name     AS freelancer_name ,
8       c.contract_id
9 FROM payments pay
10 JOIN contracts c
11     ON pay.contract_id = c.contract_id
12 JOIN bids b
13     ON c.bid_id = b.bid_id
```



```
14 JOIN projects p
15     ON b.project_id = p.project_id
16 JOIN users uf
17     ON b.freelancer_id = uf.user_id
18 WHERE p.client_id = :user_id
19 ORDER BY pay.pay_date DESC
```

Listing 4.16: Query 11b: Payments for Freelancer

```
1 SELECT pay.payment_id ,
2         pay.amount ,
3         pay.pay_date ,
4         pay.method ,
5         pay.status ,
6         p.title      AS project_title ,
7         uc.name      AS client_name ,
8         c.contract_id
9 FROM payments pay
10 JOIN contracts c
11     ON pay.contract_id = c.contract_id
12 JOIN bids b
13     ON c.bid_id = b.bid_id
14 JOIN projects p
15     ON b.project_id = p.project_id
16 JOIN users uc
17     ON p.client_id = uc.user_id
18 WHERE b.freelancer_id = :user_id
19 ORDER BY pay.pay_date DESC
```

The JOIN chain for both versions follows the natural path through the database schema. A payment belongs to a contract. A contract belongs to a bid. A bid belongs to a project. A project belongs to a client. This chain must be followed in full because there is no direct foreign key from payments to projects or from payments to users.

4.6 Reviews Queries

The reviews route file contains three operations. Two are SELECT queries and one is a transaction covered in Chapter 5.

4.6.1 Get Reviews Received

This query returns all reviews where the logged in user is the reviewee.

Listing 4.17: Query 12: Select Reviews Received

```
1 SELECT r.review_id ,
2         r.rating ,
3         r.comment_r ,
4         r.created_at ,
5         u.name      AS reviewer_name ,
6         u.user_id AS reviewer_id ,
```

```
7      p.title    AS project_title
8 FROM reviews r
9 JOIN users u
10    ON r.reviewer_id = u.user_id
11 JOIN contracts c
12    ON r.contract_id = c.contract_id
13 JOIN bids b
14    ON c.bid_id = b.bid_id
15 JOIN projects p
16    ON b.project_id = p.project_id
17 WHERE r.reviewee_id = :user_id
18 ORDER BY r.created_at DESC
```

4.6.2 Get Reviews Given

This query returns all reviews where the logged in user is the reviewer. The structure is identical to the received query except the JOIN with users is on reviewee_id instead of reviewer_id, and the WHERE clause filters on reviewer_id instead of reviewee_id.

Listing 4.18: Query 13: Select Reviews Given

```
1 SELECT r.review_id ,
2       r.rating ,
3       r.comment_r ,
4       r.created_at ,
5       u.name      AS reviewee_name ,
6       u.user_id AS reviewee_id ,
7       p.title     AS project_title
8 FROM reviews r
9 JOIN users u
10    ON r.reviewee_id = u.user_id
11 JOIN contracts c
12    ON r.contract_id = c.contract_id
13 JOIN bids b
14    ON c.bid_id = b.bid_id
15 JOIN projects p
16    ON b.project_id = p.project_id
17 WHERE r.reviewer_id = :user_id
18 ORDER BY r.created_at DESC
```

4.7 Disputes Queries

The disputes route file contains three operations. Two are straightforward and covered here. The INSERT is part of a validation sequence explained below.

4.7.1 Get Disputes for Logged In User

This query returns all disputes where the logged in user is either the client or the freelancer on the related contract.

Listing 4.19: Query 14: Select Disputes for User

```
1 SELECT d.dispute_id,
2       d.reason,
3       d.status,
4       d.created_at,
5       u.name      AS raised_by_name,
6       p.title     AS project_title,
7       c.contract_id,
8       c.amount
9 FROM disputes d
10 JOIN users u
11     ON d.raised_by = u.user_id
12 JOIN contracts c
13     ON d.contract_id = c.contract_id
14 JOIN bids b
15     ON c.bid_id = b.bid_id
16 JOIN projects p
17     ON b.project_id = p.project_id
18 WHERE p.client_id = :user_id
19 OR    b.freelancer_id = :user_id
20 ORDER BY d.created_at DESC
```

The OR condition in the WHERE clause is what makes this query return disputes for both sides of a contract. A dispute is visible to the client if `p.client_id` matches their `user_id`, and visible to the freelancer if `b.freelancer_id` matches. Both bind variables reference the same `:user_id` value.

4.7.2 Raise a Dispute

Before inserting a dispute, two validation queries run. The first confirms the contract exists and the user is part of it.

Listing 4.20: Query 15a: Verify Contract and User

```
1 SELECT c.contract_id, c.status
2 FROM contracts c
3 JOIN bids b
4     ON c.bid_id = b.bid_id
5 JOIN projects p
6     ON b.project_id = p.project_id
7 WHERE c.contract_id = :id
8 AND (
9     p.client_id = :user_id
10    OR b.freelancer_id = :user_id
11 )
```

The second checks that no open dispute already exists for this contract.

Listing 4.21: Query 15b: Check for Existing Open Dispute

```
1 SELECT dispute_id
2 FROM disputes
```

```
3 WHERE contract_id = :contract_id
4 AND    status      = 'open'
```

If both checks pass the dispute is inserted.

Listing 4.22: Query 15c: Insert New Dispute

```
1 INSERT INTO disputes
2 VALUES (
3     seq_disputes.nextval,
4     :contract_id,
5     :raised_by,
6     :reason,
7     'open',
8     sysdate
9 )
```

4.7.3 Resolve a Dispute

Resolving a dispute is a single UPDATE that changes the status from open to resolved.

Listing 4.23: Query 16: Update Dispute Status

```
1 UPDATE disputes
2 SET    status = 'resolved'
3 WHERE dispute_id = :id
```

4.8 Notifications Queries

The notifications route file contains four operations covering SELECT and UPDATE queries on the notifications table.

4.8.1 Get All Notifications for User

Listing 4.24: Query 17: Select Notifications for User

```
1 SELECT notification_id,
2         message,
3         date_sent,
4         is_read,
5         type
6 FROM notifications
7 WHERE user_id = :user_id
8 ORDER BY date_sent DESC
```

This is the only query in the user facing routes that involves just one table with no JOIN. The notifications table stores all the information needed directly.

4.8.2 Get Unread Count

This query returns a single number representing how many unread notifications the logged in user has. It is used by the dashboard to display the unread alert count.

Listing 4.25: Query 18: Count Unread Notifications

```

1 SELECT COUNT(*)
2 FROM notifications
3 WHERE user_id = :user_id
4 AND    is_read = 'N'

```

4.8.3 Mark Single Notification as Read

Listing 4.26: Query 19: Mark One Notification Read

```

1 UPDATE notifications
2 SET    is_read = 'Y'
3 WHERE notification_id = :id
4 AND    user_id      = :user_id

```

The AND user_id = :user_id condition ensures a user can only mark their own notifications as read. Without this condition, any authenticated user could mark any notification as read by guessing the notification ID.

4.8.4 Mark All Notifications as Read

Listing 4.27: Query 20: Mark All Notifications Read

```

1 UPDATE notifications
2 SET    is_read = 'Y'
3 WHERE user_id = :user_id

```

This query updates every row for the logged in user in a single statement rather than looping through each notification individually.

4.9 Summary of User Facing Queries

Query	Operation	Route	Tables
1	SELECT	GET /projects	projects, users, categories, bids
2	SELECT	GET /projects/client/:id	projects, categories, bids
3	SELECT	GET /projects/:id	projects, users, categories
4	INSERT	POST /projects	projects
5a	SELECT	DELETE /projects/:id	bids
5b	DELETE	DELETE /projects/:id	projects
6	SELECT	GET /bids/project/:id	bids, users, freelancer_profile
7	SELECT	GET /bids/freelancer/:id	bids, projects, users
8a	SELECT	POST /bids	projects

Query	Operation	Route	Tables
8b	SELECT	POST /bids	bids
8c	INSERT	POST /bids	bids
9a	SELECT	GET /contracts/my	contracts, bids, projects, users
9b	SELECT	GET /contracts/my	contracts, bids, projects, users
10	SELECT	GET /contracts/:id	contracts, bids, projects, users
11a	SELECT	GET /payments/my	payments, contracts, bids, projects, users
11b	SELECT	GET /payments/my	payments, contracts, bids, projects, users
12	SELECT	GET /reviews/my	reviews, users, contracts, bids, projects
13	SELECT	GET /reviews/given	reviews, users, contracts, bids, projects
14	SELECT	GET /disputes/my	disputes, users, contracts, bids, projects
15a	SELECT	POST /disputes	contracts, bids, projects
15b	SELECT	POST /disputes	disputes
15c	INSERT	POST /disputes	disputes
16	UPDATE	PUT /disputes/:id/resolve	disputes
17	SELECT	GET /notifications/my	notifications
18	SELECT	GET /notifications/unread-count	notifications
19	UPDATE	PUT /notifications/:id/read	notifications
20	UPDATE	PUT /notifications/read-all	notifications

Table 4.3: Complete Summary of User Facing Queries

4.10 Summary

This chapter covered all twenty queries used in the user facing routes. The projects queries use LEFT JOINS and correlated subqueries to count bids. The bids queries join three tables and use two validation checks before every insert. The contracts queries branch by role and use table aliases to join users twice in a single query. The payments queries follow a five table JOIN chain because there is no direct foreign key from payments to users. The reviews queries join five tables to get both the reviewer name and the project title. The disputes query uses an OR condition in the WHERE clause to return disputes for both sides of a contract. The notifications queries are the simplest in the application, working mostly against a single table. The following chapter covers the three transactions that keep the database consistent when multiple tables must be updated together.

Chapter 5

Transactions

5.1 Overview

A transaction is a sequence of SQL statements that must all succeed together or all fail together. If any statement in the middle of a transaction fails, every statement that already ran is rolled back as if it never happened. The database is left exactly as it was before the transaction began.

SkillLink has three operations that require transactions. The first is user registration, which inserts into two tables. The second is accepting a bid, which modifies four tables and inserts into two more. The third is recording a payment, which inserts a payment row and updates two other tables. All three use the same pattern in `oracledb`: every `execute` call inside the transaction passes `autoCommit: false`, and a single `connection.commit()` call at the end writes all changes permanently.

5.2 How autoCommit Works in oracledb

By default, `oracledb` must be explicitly told whether to commit after each statement. SkillLink passes `autoCommit: false` as the third argument to every `connection.execute` call that belongs to a transaction. Statements that stand alone and do not need to be grouped with others pass `autoCommit: true`, which commits immediately after that single statement executes.

Listing 5.1: `autoCommit false` vs `true`

```
1 // Part of a transaction - does NOT commit yet
2 await connection.execute(
3   'UPDATE bids SET status = 'accepted' WHERE bid_id =
4     :bid_id',
5   { bid_id },
6   { autoCommit: false }
7 );
8 // Standalone insert - commits immediately
9 await connection.execute(
10  'INSERT INTO bids VALUES (seq_bids.nextval, ...)',
```

```
11     { ... },  
12     { autoCommit: true }  
13 );
```

When all statements in a transaction have run without error, a single commit call writes every pending change to the database at once.

Listing 5.2: Committing and Closing After a Transaction

```
1 await connection.commit();  
2 await connection.close();
```

If an error is thrown at any point before the commit, the `catch` block is entered. Because no commit was ever issued, Oracle automatically discards all uncommitted changes when the connection is closed or the session ends. The database is left in exactly the state it was in before the transaction began.

5.3 Transaction 1: User Registration

Registration is the simplest transaction in SkillLink. It groups two INSERT statements so that a user row is never created without a matching profile row, and a profile row is never created without a matching user row.

5.3.1 Why a Transaction Is Needed

Without a transaction, the server could insert into the users table successfully and then crash or encounter a constraint error before inserting into the profile table. The result would be a user who can never log in because the login query joins users with the profile tables and the CASE WHEN expression would return NULL for the role. Wrapping both inserts in one transaction prevents this half-written state entirely.

5.3.2 Step 1: Insert into Users

Listing 5.3: Registration Step 1: Insert User Row

```
1 INSERT INTO users  
2 VALUES (  
3     seq_users.nextval,  
4     :name,  
5     :email,  
6     :password,  
7     :phone,  
8     sysdate  
9 )  
10 RETURNING user_id INTO :user_id
```

The RETURNING clause sends the newly generated primary key back to the backend immediately. This value is used as the primary key for the profile insert in the next step. Without RETURNING a second SELECT query would be needed to find the ID that was just created.

5.3.3 Step 2: Insert into Profile Table

Listing 5.4: Registration Step 2a: Insert Client Profile

```
1 INSERT INTO client_profile
2 VALUES (:id, :company_name, :location)
```

Listing 5.5: Registration Step 2b: Insert Freelancer Profile

```
1 INSERT INTO freelancer_profile
2 VALUES (
3     :id,
4     :bio,
5     :hourly_rate,
6     0,
7     :experience_level,
8     'yes'
9 )
```

The `:id` bind variable in both profile inserts is the `user_id` returned from the first insert. The rating is hardcoded to 0 and availability to 'yes' because every new freelancer starts with no reviews and is available by default.

5.3.4 Step 3: Commit

Listing 5.6: Registration Step 3: Commit Transaction

```
1 await connection.commit();
2 await connection.close();
```

Step	SQL	Table	autoCommit
1	INSERT with RETURNING	users	false
2	INSERT profile row	client_profile or freelancer_profile	false
3	COMMIT	both	—

Table 5.1: Registration Transaction Steps

5.4 Transaction 2: Accept Bid

Accepting a bid is the most complex transaction in SkillLink. When a client accepts one bid, five SQL statements must all succeed together: the accepted bid is updated, all other bids on the same project are rejected, the project status is changed, a new contract row is inserted, and a notification is sent to the freelancer.

If any of these five statements fails, none of the others should take effect. A contract must never exist without the bid being marked accepted. The project must never be marked in progress without a contract being created.

5.4.1 Step 1: Fetch the Bid

Before the transaction begins, a SELECT query retrieves the bid details needed for the subsequent statements.

Listing 5.7: Accept Bid Pre-Check: Fetch Bid Details

```
1 SELECT b.bid_id ,
2       b.project_id ,
3       b.freelancer_id ,
4       b.amount ,
5       b.delivery_time
6 FROM bids b
7 JOIN projects p
8   ON b.project_id = p.project_id
9 WHERE b.bid_id     = :bid_id
10 AND   p.client_id = :client_id
```

The JOIN with projects and the AND p.client_id = :client_id condition ensure that only the client who owns the project can accept a bid on it. If the bid does not exist or belongs to a different client's project, the query returns no rows and the operation is rejected before any changes are made.

5.4.2 Step 2: Accept the Selected Bid

Listing 5.8: Accept Bid Step 2: Mark Bid as Accepted

```
1 UPDATE bids
2 SET   status = 'accepted'
3 WHERE bid_id = :bid_id
```

5.4.3 Step 3: Reject All Other Bids on the Same Project

Listing 5.9: Accept Bid Step 3: Reject Remaining Bids

```
1 UPDATE bids
2 SET   status = 'rejected'
3 WHERE project_id = :project_id
4 AND   bid_id    != :bid_id
```

The NOT EQUAL condition bid_id != :bid_id ensures the accepted bid is not accidentally overwritten by this second UPDATE. Every other bid on the same project is moved to rejected status in a single statement regardless of how many bids exist.

5.4.4 Step 4: Update Project Status

Listing 5.10: Accept Bid Step 4: Set Project to in_progress

```
1 UPDATE projects
2 SET   status = 'in_progress'
3 WHERE project_id = :project_id
```

This prevents any further bids from being placed on the project. The bid submission route checks that the project status is 'open' before allowing an insert. Once the project is in-progress that check will fail for any new bid attempt.

5.4.5 Step 5: Create the Contract

The contract end date is calculated in JavaScript by adding the `delivery_time` number of days to the current date. The result is formatted as a YYYY-MM-DD string and passed to Oracle's `TO_DATE` function.

Listing 5.11: Calculating Contract End Date in JavaScript

```
1 const endDate = new Date();
2 endDate.setDate(endDate.getDate() + Number(delivery_time));
3 const endDateStr = endDate.toISOString().split('T')[0];
```

Listing 5.12: Accept Bid Step 5: Insert Contract Row

```
1 INSERT INTO contracts
2 VALUES (
3     seq_contracts.nextval,
4     :bid_id,
5     :amount,
6     sysdate,
7     TO_DATE(:end_date, 'YYYY-MM-DD'),
8     'active'
9 )
```

The `bid_id` is the foreign key that links the contract back to the bid, and through the bid, to the project and the freelancer. There is no direct `project_id` or `freelancer_id` column in the `contracts` table. All joins to those entities go through the `bids` table.

5.4.6 Step 6: Insert Notification

Listing 5.13: Accept Bid Step 6: Notify Freelancer

```
1 INSERT INTO notifications
2 VALUES (
3     seq_notifications.nextval,
4     :user_id,
5     'Your bid has been accepted. A contract has been created.',
6     sysdate,
7     'N',
8     'bid_accepted'
9 )
```

5.4.7 Step 7: Commit

Listing 5.14: Accept Bid Step 7: Commit Transaction

```
1 await connection.commit();
2 await connection.close();
```

Step	SQL	Table	Rows Affected	autoCommit
Pre-check	SELECT	bids, projects	1	—
2	UPDATE	bids	1 (accepted)	false
3	UPDATE	bids	N (rejected)	false
4	UPDATE	projects	1	false
5	INSERT	contracts	1	false
6	INSERT	notifications	1	false
7	COMMIT	all	—	—

Table 5.2: Accept Bid Transaction Steps

5.5 Transaction 3: Record Payment

Recording a payment is the final significant transaction. When a client pays a freelancer, three things must happen together: the payment row is inserted, the contract is marked completed, the project is marked completed, and the freelancer receives a notification.

5.5.1 Step 1: Verify the Contract

Before the transaction begins, a SELECT confirms the contract exists, belongs to the logged in client, and is not already completed.

Listing 5.15: Record Payment Pre-Check: Verify Contract

```

1 SELECT c.contract_id ,
2       c.status ,
3       b.freelancer_id
4 FROM contracts c
5 JOIN bids b
6   ON c.bid_id = b.bid_id
7 JOIN projects p
8   ON b.project_id = p.project_id
9 WHERE c.contract_id = :id
10 AND   p.client_id   = :client_id

```

If the returned status is already 'completed' the payment is rejected with a 400 error. This prevents a client from making duplicate payments on the same contract.

5.5.2 Step 2: Insert the Payment Row

Listing 5.16: Record Payment Step 2: Insert Payment

```

1 INSERT INTO payments
2 VALUES (
3   seq_payments.nextval ,
4   :contract_id ,
5   :amount ,
6   sysdate ,
7   :method ,
8   'paid'

```

```
9 | )
```

The method is validated by the backend before the query runs. Only the four values 'easy-paisa', 'jazzcash', 'bank.transfer', and 'cash' are accepted. Any other value is rejected before the database is touched.

5.5.3 Step 3: Mark Contract as Completed

Listing 5.17: Record Payment Step 3: Complete Contract

```
1 UPDATE contracts
2 SET   status = 'completed'
3 WHERE contract_id = :id
```

5.5.4 Step 4: Mark Project as Completed

The project has no direct foreign key to the contracts table. The backend traces the relationship through a subquery that walks the join chain from contracts through bids to projects.

Listing 5.18: Record Payment Step 4: Complete Project

```
1 UPDATE projects
2 SET   status = 'completed'
3 WHERE project_id = (
4     SELECT p.project_id
5     FROM   projects p
6     JOIN   bids b
7           ON b.project_id = p.project_id
8     JOIN   contracts c
9           ON c.bid_id = b.bid_id
10    WHERE  c.contract_id = :id
11 )
```

5.5.5 Step 5: Notify the Freelancer

Listing 5.19: Record Payment Step 5: Notify Freelancer

```
1 INSERT INTO notifications
2 VALUES (
3     seq_notifications.nextval,
4     :user_id,
5     'Payment of Rs.' || :amount || ' has been received for your
        contract.',
6     sysdate,
7     'N',
8     'payment'
9 )
```

5.5.6 Step 6: Commit

Listing 5.20: Record Payment Step 6: Commit Transaction

```
1 await connection.commit();
2 await connection.close();
```

Step	SQL	Table	Rows Affected	autoCommit
Pre-check	SELECT	contracts, bids, projects	1	—
2	INSERT	payments	1	false
3	UPDATE	contracts	1	false
4	UPDATE	projects	1 (via subquery)	false
5	INSERT	notifications	1	false
6	COMMIT	all	—	—

Table 5.3: Record Payment Transaction Steps

5.6 What Happens on Failure

If any statement throws an error before the commit, the JavaScript `catch` block is entered. The connection is closed without a commit having been issued.

Listing 5.21: Error Handling in Transactions

```
1 try {
2   const connection = await getConnection();
3
4   await connection.execute('UPDATE bids ...', { ... }, {
5     autoCommit: false });
6   await connection.execute('UPDATE bids ...', { ... }, {
7     autoCommit: false });
8   await connection.execute('UPDATE projects ...',{ ... }, {
9     autoCommit: false });
10  await connection.execute('INSERT INTO contracts ...', { ...
11    }, { autoCommit: false });
12  await connection.execute('INSERT INTO notifications ...', {
13    ... }, { autoCommit: false });
14
15  await connection.commit();
16  await connection.close();
17
18  res.json({ message: 'Bid accepted and contract created
19    successfully' });
20 } catch (err) {
21   // connection.close() discards all uncommitted changes
22   // automatically
23   res.status(500).json({ error: 'Failed to accept bid',
24     details: err.message });
25 }
```

Oracle's transaction model guarantees atomicity. When a connection is closed with uncommitted changes, those changes are rolled back. No partial state can persist. A contract row can never exist in the database without the corresponding bid being marked accepted. A payment row can never exist without the contract being completed.

5.7 Summary

SkillLink uses three database transactions to maintain consistency across multiple tables. The registration transaction groups two inserts so a user is always created with a profile. The accept bid transaction coordinates five SQL statements across four tables so a contract is always created alongside the accepted bid and the rejected bids in a single atomic operation. The record payment transaction links a payment insert to contract and project status updates so the completed state is always consistent. All three transactions use the same `autoCommit: false` pattern with a single `connection.commit()` at the end. Errors before the commit leave the database unchanged. The following chapter covers the admin queries which provide a full view of all data across every table in the system.

Chapter 6

Admin Queries

6.1 Overview

The admin route file provides ten endpoints that give the platform administrator a complete view of every table in the SkillLink database. Every endpoint is protected by two middleware functions: `verifyToken` verifies the JWT token and `adminOnly` checks that the role in that token is exactly 'admin'. A client or freelancer token cannot access any of these endpoints even if the token is otherwise valid.

All queries in this chapter are read-only `SELECT` statements. The admin panel is a monitoring and oversight interface. It does not insert, update, or delete any data. Every query returns all rows ordered by the primary key descending so the most recently created records appear first.

6.2 Dashboard Stats

The stats endpoint is the first thing the admin sees after login. It runs nine separate `COUNT` queries in parallel using `Promise.all` and assembles the results into a single JSON response. Each query targets one table and returns a single number.

6.2.1 The Nine Count Queries

Listing 6.1: Stats Query 1: Total Users

```
1 SELECT COUNT(*)  
2 FROM users
```

Listing 6.2: Stats Query 2: Total Projects

```
1 SELECT COUNT(*)  
2 FROM projects
```

Listing 6.3: Stats Query 3: Total Bids

```
1 SELECT COUNT(*)  
2 FROM bids
```


Listing 6.4: Stats Query 4: Total Contracts

```
1 SELECT COUNT(*)
2 FROM contracts
```

Listing 6.5: Stats Query 5: Total Payments

```
1 SELECT COUNT(*)
2 FROM payments
```

Listing 6.6: Stats Query 6: Total Disputes

```
1 SELECT COUNT(*)
2 FROM disputes
```

Listing 6.7: Stats Query 7: Total Revenue from Paid Payments

```
1 SELECT NVL(SUM(amount), 0)
2 FROM payments
3 WHERE status = 'paid'
```

The NVL function wraps the SUM so that if no paid payments exist yet the query returns 0 instead of NULL. A NULL revenue value would cause the frontend to display nothing rather than a zero, which would be confusing on a fresh installation.

Listing 6.8: Stats Query 8: Total Clients

```
1 SELECT COUNT(*)
2 FROM client_profile
```

Listing 6.9: Stats Query 9: Total Freelancers

```
1 SELECT COUNT(*)
2 FROM freelancer_profile
```

Clients and freelancers are counted from their respective profile tables rather than from the users table because the users table also contains admin accounts. Counting from the profile tables gives exact role-specific numbers.

6.2.2 How Promise.all Is Used

The six core counts run simultaneously using `Promise.all`. The revenue and profile counts run as sequential awaits after the parallel group resolves. This reduces the total waiting time because multiple Oracle queries execute at the same time over separate connections borrowed from the pool.

Listing 6.10: Running Stats Queries in Parallel

```
1 const [users, projects, bids, contracts, payments, disputes] =
2   await Promise.all([
3     connection.execute('SELECT COUNT(*) FROM users'),
4     connection.execute('SELECT COUNT(*) FROM projects'),
5     connection.execute('SELECT COUNT(*) FROM bids'),
```

```

6      connection.execute('SELECT COUNT(*) FROM contracts '),
7      connection.execute('SELECT COUNT(*) FROM payments '),
8      connection.execute('SELECT COUNT(*) FROM disputes '),
9  ]);

```

Response Field	Query	Table
users	COUNT(*)	users
clients	COUNT(*)	client_profile
freelancers	COUNT(*)	freelancer_profile
projects	COUNT(*)	projects
bids	COUNT(*)	bids
contracts	COUNT(*)	contracts
payments	COUNT(*)	payments
disputes	COUNT(*)	disputes
revenue	NVL(SUM(amount), 0) WHERE status = 'paid'	payments

Table 6.1: Dashboard Stats Fields and Their Source Queries

6.3 All Users

The users endpoint returns every registered user on the platform along with their role and the relevant profile data for that role. It is the most complex admin SELECT because it must show clients and freelancers together in a single result set even though their profile data lives in different tables.

Listing 6.11: Admin Query: All Users with Role and Profile Data

```

1  SELECT u.user_id ,
2         u.name ,
3         u.email ,
4         u.phone ,
5         u.created_at ,
6         CASE
7             WHEN c.client_id      IS NOT NULL THEN 'client'
8             WHEN f.freelancer_id IS NOT NULL THEN 'freelancer'
9         END AS role ,
10        c.company_name ,
11        c.location ,
12        f.hourly_rate ,
13        f.rating ,
14        f.experience_level ,
15        f.availability
16 FROM users u
17 LEFT JOIN client_profile c
18     ON u.user_id = c.client_id
19 LEFT JOIN freelancer_profile f
20     ON u.user_id = f.freelancer_id
21 ORDER BY u.user_id DESC

```

Both joins are LEFT JOINS. For a client row, all the freelancer_profile columns will be NULL and the client_profile columns will have values. For a freelancer row the opposite is true. The CASE WHEN expression checks which profile table has a non-null match and sets the role label accordingly.

This single query replaces what would otherwise require separate queries for clients and freelancers and a manual merge on the frontend.

Column	Source Table	Present For
user_id, name, email, phone, created_at	users	all users
role	CASE WHEN expression	all users
company_name, location	client_profile	clients only
hourly_rate, rating, experience_level, availability	freelancer_profile	freelancers only

Table 6.2: Column Sources for Admin Users Query

6.4 All Projects

The projects endpoint returns every project on the platform with the client name, company name, category, and a live bid count calculated by a correlated subquery.

Listing 6.12: Admin Query: All Projects

```

1 SELECT p.project_id,
2       p.title,
3       p.description,
4       p.budget,
5       p.deadline,
6       p.status,
7       u.name          AS client_name,
8       c.company_name,
9       cat.name        AS category,
10      (SELECT COUNT(*)
11       FROM bids b
12       WHERE b.project_id = p.project_id) AS bid_count
13 FROM projects p
14 JOIN client_profile c
15     ON p.client_id = c.client_id
16 JOIN users u
17     ON c.client_id = u.user_id
18 LEFT JOIN project_category pc
19     ON p.project_id = pc.project_id
20 LEFT JOIN categories cat
21     ON pc.category_id = cat.category_id
22 ORDER BY p.project_id DESC

```

The JOIN chain here is slightly different from the user-facing projects query. The admin version joins through client_profile first to get the company_name, then joins users to get the client name. The user-facing version joins users directly. Both produce the same

client name but the admin version additionally surfaces the company name that is stored in the client_profile table.

The two LEFT JOINs for project_category and categories handle projects that were posted without a category. Such projects would disappear from the results if INNER JOINs were used.

6.5 All Bids

The bids endpoint returns every bid across all projects with the project title, budget, freelancer name, and experience level joined in from three tables.

Listing 6.13: Admin Query: All Bids

```

1 SELECT b.bid_id,
2       b.amount,
3       b.delivery_time,
4       b.status,
5       b.submitted_at,
6       p.title           AS project_title,
7       p.budget,
8       u.name            AS freelancer_name,
9       f.experience_level
10 FROM bids b
11 JOIN projects p
12     ON b.project_id = p.project_id
13 JOIN freelancer_profile f
14     ON b.freelancer_id = f.freelancer_id
15 JOIN users u
16     ON f.freelancer_id = u.user_id
17 ORDER BY b.bid_id DESC

```

The join path from bids to the freelancer name goes through freelancer_profile first and then to users. This is because the freelancer_id on the bids table is a foreign key to freelancer_profile, not directly to users. The users join then retrieves the name using the same ID since freelancer_id and user_id share the same value for a given freelancer.

Column	Source Table
bid_id, amount, delivery_time, status, submitted_at	bids
project_title, budget	projects
freelancer_name	users
experience_level	freelancer_profile

Table 6.3: Column Sources for Admin Bids Query

6.6 All Contracts

The contracts endpoint returns every contract with the project title, client name, and freelancer name. Because a contract has no direct foreign key to the users table, the

join chain must pass through bids and projects to reach both parties. The users table is joined twice using two different aliases.

Listing 6.14: Admin Query: All Contracts

```

1 SELECT con.contract_id,
2         con.amount,
3         con.start_date,
4         con.end_date,
5         con.status,
6         p.title      AS project_title,
7         uc.name      AS client_name,
8         uf.name      AS freelancer_name
9 FROM contracts con
10 JOIN bids b
11     ON con.bid_id = b.bid_id
12 JOIN projects p
13     ON b.project_id = p.project_id
14 JOIN client_profile c
15     ON p.client_id = c.client_id
16 JOIN users uc
17     ON c.client_id = uc.user_id
18 JOIN freelancer_profile f
19     ON b.freelancer_id = f.freelancer_id
20 JOIN users uf
21     ON f.freelancer_id = uf.user_id
22 ORDER BY con.contract_id DESC

```

The alias `uc` represents the client and is reached by following contracts to bids, bids to projects, projects to client_profile, and client_profile to users. The alias `uf` represents the freelancer and is reached by following bids to freelancer_profile and freelancer_profile to users. Both paths end at the users table but through completely different foreign key chains.

Join Path	Purpose
contracts → bids	Link contract to its originating bid
bids → projects	Get project title
projects → client_profile → users uc	Get client name
bids → freelancer_profile → users uf	Get freelancer name

Table 6.4: Join Path Breakdown for Admin Contracts Query

6.7 All Payments

The payments endpoint returns every payment with the project title, client name, and freelancer name. The join chain is one step longer than contracts because payments must first join to contracts before following the same path.

Listing 6.15: Admin Query: All Payments

```
1 SELECT pay.payment_id ,
2         pay.amount ,
3         pay.pay_date ,
4         pay.method ,
5         pay.status ,
6         con.contract_id ,
7         uf.name      AS freelancer_name ,
8         p.title      AS project_title ,
9         uc.name      AS client_name
10 FROM payments pay
11 JOIN contracts con
12     ON pay.contract_id = con.contract_id
13 JOIN bids b
14     ON con.bid_id = b.bid_id
15 JOIN projects p
16     ON b.project_id = p.project_id
17 JOIN client_profile c
18     ON p.client_id = c.client_id
19 JOIN users uc
20     ON c.client_id = uc.user_id
21 JOIN freelancer_profile f
22     ON b.freelancer_id = f.freelancer_id
23 JOIN users uf
24     ON f.freelancer_id = uf.user_id
25 ORDER BY pay.payment_id DESC
```

This query joins seven tables. There is no shortcut available because the database schema has no direct foreign key between payments and users. The only path from a payment to the people involved passes through contracts, bids, projects, and the two profile tables. The same dual-alias pattern for uc and uf is applied here as in the contracts query.

6.8 All Reviews

The reviews endpoint returns every review with the reviewer name, reviewee name, and project title. The users table is again joined twice under two different aliases to retrieve names for both sides of the review.

Listing 6.16: Admin Query: All Reviews

```
1 SELECT r.review_id ,
2         r.rating ,
3         r.comment_r ,
4         r.created_at ,
5         u1.name      AS reviewer_name ,
6         u2.name      AS reviewee_name ,
7         p.title      AS project_title
8 FROM reviews r
9 JOIN users u1
10     ON r.reviewer_id = u1.user_id
11 JOIN users u2
12     ON r.reviewee_id = u2.user_id
```

```
13 JOIN contracts con
14     ON r.contract_id = con.contract_id
15 JOIN bids b
16     ON con.bid_id = b.bid_id
17 JOIN projects p
18     ON b.project_id = p.project_id
19 ORDER BY r.review_id DESC
```

The reviews table stores two user IDs: `reviewer_id` and `reviewee_id`. To get both names in a single query, users is joined twice. The alias `u1` resolves the reviewer name and `u2` resolves the reviewee name. The project title requires traversing reviews to contracts, contracts to bids, and bids to projects because the reviews table stores only the `contract_id` and has no direct link to projects.

6.9 All Skills

The skills endpoint returns every skill assigned to every freelancer. It joins four tables to combine freelancer profile data with the skill name and category from the skills lookup table.

Listing 6.17: Admin Query: All Freelancer Skills

```
1 SELECT u.name          AS freelancer_name ,
2       f.hourly_rate ,
3       f.experience_level ,
4       f.availability ,
5       s.name           AS skill ,
6       s.category       AS skill_category
7 FROM users u
8 JOIN freelancer_profile f
9     ON u.user_id = f.freelancer_id
10 JOIN user_skills us
11     ON f.freelancer_id = us.user_id
12 JOIN skills s
13     ON us.skill_id = s.skill_id
14 ORDER BY u.name
```

The `user_skills` table is a junction table that resolves the many-to-many relationship between freelancers and skills. One freelancer can have many skills and one skill can belong to many freelancers. This query returns one row per skill per freelancer, so a freelancer with four skills will appear as four rows in the result. The results are ordered by freelancer name alphabetically rather than by ID so the admin sees all skills for one freelancer grouped together.

6.10 All Disputes

The disputes endpoint returns every dispute raised on the platform with the name of the user who raised it, the associated contract, and the project title.

Table	Role in Query
users	Freelancer name
freelancer_profile	Hourly rate, experience level, availability
user_skills	Junction table resolving many-to-many relationship
skills	Skill name and category label

Table 6.5: Table Roles in the Admin Skills Query

Listing 6.18: Admin Query: All Disputes

```

1 SELECT d.dispute_id,
2       d.reason,
3       d.status,
4       d.created_at,
5       u.name           AS raised_by,
6       con.contract_id,
7       con.amount,
8       p.title          AS project_title
9 FROM disputes d
10 JOIN users u
11     ON d.raised_by = u.user_id
12 JOIN contracts con
13     ON d.contract_id = con.contract_id
14 JOIN bids b
15     ON con.bid_id = b.bid_id
16 JOIN projects p
17     ON b.project_id = p.project_id
18 ORDER BY d.dispute_id DESC

```

The disputes table stores the user_id of whoever raised the dispute in the raised_by column. The JOIN with users on that column resolves the name. The contract amount is included so the admin can immediately see the financial value of the dispute without navigating to the contracts view. The project title is reached by following the same contracts to bids to projects chain used in several other admin queries.

6.11 All Notifications

The notifications endpoint returns every notification sent to every user on the platform. It is the only admin query that involves just one JOIN. The notifications table stores the user_id directly so only one join to users is needed to resolve the recipient name.

Listing 6.19: Admin Query: All Notifications

```

1 SELECT n.notification_id,
2       n.message,
3       n.date_sent,
4       n.is_read,
5       n.type,
6       u.name      AS user_name
7 FROM notifications n

```



```

8 JOIN users u
9   ON n.user_id = u.user_id
10 ORDER BY n.date_sent DESC

```

The type column identifies which system event generated the notification. The possible values are 'bid_accepted', 'payment', and 'review', corresponding to the three places in the backend that insert notification rows: the accept bid transaction, the record payment transaction, and the submit review operation. The is_read column uses 'Y' and 'N' rather than a boolean because Oracle's VARCHAR2 type was used for this flag in the schema design.

6.12 Summary of Admin Queries

Endpoint	Tables Joined	Notable Features
GET /admin/stats	7 individual tables	9 queries via Proc
GET /admin/users	users, client_profile, freelancer_profile	CASE WHEN fo
GET /admin/projects	projects, users, client_profile, categories, bids	Correlated subqu
GET /admin/bids	bids, projects, freelancer_profile, users	Join through pro
GET /admin/contracts	contracts, bids, projects, users uc, users uf	users joined twic
GET /admin/payments	payments, contracts, bids, projects, users uc, users uf	7-table join, dual
GET /admin/reviews	reviews, users u1, users u2, contracts, bids, projects	users joined twic
GET /admin/skills	users, freelancer_profile, user_skills, skills	Junction table fo
GET /admin/disputes	disputes, users, contracts, bids, projects	raised by resolve
GET /admin/notifications	notifications, users	Simplest admin j

Table 6.6: Summary of All Admin Queries

6.13 Summary

The ten admin endpoints give the administrator a complete read-only view of the entire SkillLink database. The stats endpoint runs nine COUNT queries in parallel to build the dashboard numbers. The users query uses a dual LEFT JOIN with a CASE WHEN expression to show clients and freelancers in a single result set. The contracts, payments, and reviews queries all use the dual-alias pattern to join the users table twice and retrieve names for both parties on a contract. The skills query uses the user_skills junction table to resolve the many-to-many relationship between freelancers and skills. Every query is protected by both the verifyToken middleware and the adminOnly role check. The following chapter provides a complete API reference listing every endpoint in the system.

Chapter 7

API Reference

7.1 Overview

This chapter provides a complete reference for every HTTP endpoint in the SkillLink backend. Each endpoint is listed with its HTTP method, full URL path, whether authentication is required, which role is permitted to call it, the Oracle tables it touches, and a description of what it does.

The base URL for all endpoints is `http://localhost:5000/api`. Every request and response body uses JSON. Protected endpoints require the Authorization header to be set in the following format.

Listing 7.1: Authorization Header Format

```
1 Authorization: Bearer <token>
```

The token is the JWT string returned by the login or register endpoint. It expires after seven days.

7.2 Authentication Endpoints

These endpoints handle user registration, user login, and admin login. They do not require a token. They are the only endpoints that insert into the users, client_profile, and freelancer_profile tables.

Method	URL	Auth	Role	Tables	Description
POST	/auth/register	No	Any	users, client_profile or freelancer_profile	Registers a new user. Inserts into users and the matching profile table inside one transaction. Returns a JWT token.

Method	URL	Auth	Role	Tables	Description
POST	/auth/login	No	Any	users, client_profile, free-lancer_profile	Authenticates an existing user. Runs a three-table LEFT JOIN with CASE WHEN to determine role. Returns a JWT token.
POST	/admin/auth/login	No	Admin	admins	Authenticates an admin. Queries the admins table only. Returns a JWT token with role hardcoded as admin.

Table 7.1: Authentication Endpoints

7.2.1 Register Request Body

Listing 7.2: POST /auth/register Request Body

```

1 {
2   "name":           "Ali Hassan",
3   "email":          "ali@example.com",
4   "password":       "secret123",
5   "phone":          "03001234567",    // optional
6   "role":           "client",          // "client" or
   "freelancer"
7
8   // Client only fields
9   "company_name":   "TechCorp",        // optional
10  "location":        "Lahore",          // optional
11
12  // Freelancer only fields
13  "bio":             "Full stack dev",  // optional
14  "hourly_rate":     25,                 // optional
15  "experience_level": "intermediate"    // optional
16 }
```

7.2.2 Login Request Body

Listing 7.3: POST /auth/login Request Body

```

1 {
2   "email":          "ali@example.com",
3   "password":       "secret123"
4 }
```

7.2.3 Login Response

Listing 7.4: POST /auth/login Response Body

```

1 {
2   "message": "Login successful",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
4   "user_id": 1,
5   "name": "Ali Hassan",
6   "email": "ali@example.com",
7   "role": "client"
8 }

```

7.3 Projects Endpoints

Projects are the central entity of SkillLink. The GET all projects endpoint requires no authentication so freelancers can browse without logging in. All write operations require a client token.

Method	URL	Auth	Role	Tables	Description
GET	/projects	No	Any	projects, users, categories, bids	Returns all projects on the platform. Includes client name, category, and live bid count from a correlated subquery.
GET	/projects/client/:client_id	Yes	Client	projects, categories, bids	Returns all projects posted by the specified client. Includes total bid count and pending bid count per project.
GET	/projects/:id	No	Any	projects, users, categories	Returns a single project row by primary key. Used to display the project detail page.
POST	/projects	Yes	Client	projects	Inserts a new project row. Status is hard-coded to 'open'. Deadline is converted using TO_DATE.

Method	URL	Auth	Role	Tables	Description
DELETE	/projects/:id	Yes	Client	projects, bids	Deletes a project only if it has zero bids. The client_id from the token is used in the WHERE clause to prevent deleting another client's project.

Table 7.2: Projects Endpoints

7.3.1 Post Project Request Body

Listing 7.5: POST /projects Request Body

```

1 {
2   "title":      "Build a REST API",
3   "description": "Need a Node.js backend with Oracle DB", //
4   "budget":     15000,
5   "deadline":   "2026-08-01"
6 }
```

7.4 Bids Endpoints

Bid submission is restricted to freelancers. Bid acceptance is restricted to clients. The accept operation triggers the most complex transaction in the system.

Method	URL	Auth	Role	Tables	Description
GET	/bids/project/:project_id	Yes	Client	bids, users, freelancer_profile	Returns all bids on a specific project ordered by amount ascending. Includes freelancer rating, experience level, and bio.
GET	/bids/freelancer/:freelancer_id	Yes	Freelancer	bids, projects, users	Returns all bids submitted by a specific freelancer ordered by submission date. Includes project title and client name.

Method	URL	Auth	Role	Tables	Description
POST	/bids	Yes	Freelancer	bids, projects	Submits a new bid. Validates project is open and no duplicate bid exists before inserting. freelancer_id is taken from the token, never the request body.
PUT	/bids/:bid_id/accept	Yes	Client	bids, projects, contracts, notifications	Accepts one bid and rejects all others on the same project. Updates project status to in_progress. Creates a contract. Sends a notification. All five operations run in one transaction.

Table 7.3: Bids Endpoints

7.4.1 Submit Bid Request Body

Listing 7.6: POST /bids Request Body

```

1 {
2   "project_id":    5,
3   "amount":       12000,
4   "delivery_time": 14    // number of days
5 }
```

7.5 Contracts Endpoints

The contracts route has no POST endpoint. Contracts are created automatically by the accept bid transaction. The GET endpoints serve both clients and freelancers but run different queries depending on the role in the JWT token.

Method	URL	Auth	Role	Tables	Description
GET	/contracts/my	Yes	Client or Freelancer	contracts, bids, projects, users	Returns all contracts for the logged in user. Runs a different query depending on role. For a client returns the freelancer name. For a freelancer returns the client name.
GET	/contracts/:id	Yes	Client or Freelancer	contracts, bids, projects, users uc, users uf	Returns a single contract by primary key. Joins users twice using aliases uc and uf to return both the client name and the freelancer name in one query.

Table 7.4: Contracts Endpoints

7.6 Payments Endpoints

Payments can only be recorded by clients. There is no direct payment update or delete. Recording a payment triggers a three-step transaction that also completes the contract and the project.

Method	URL	Auth	Role	Tables	Description
GET	/payments/my	Yes	Client or Freelancer	payments, contracts, bids, projects, users	Returns all payments for the logged in user. For clients shows the freelancer name. For freelancers shows the client name.

Method	URL	Auth	Role	Tables	Description
POST	/payments	Yes	Client	payments, contracts, projects, notifications	Records a new payment. Validates the contract belongs to the client and is not already completed. Marks the contract and project as completed. Notifies the freelancer. All operations run in one transaction.

Table 7.5: Payments Endpoints

7.6.1 Record Payment Request Body

Listing 7.7: POST /payments Request Body

```

1 {
2   "contract_id": 3,
3   "amount":      12000,
4   "method":      "easypaisa"
5   // method must be one of:
6   // "easypaisa", "jazzcash", "bank_transfer", "cash"
7 }

```

7.7 Reviews Endpoints

Reviews can only be submitted after a contract is completed. A user cannot review the same person twice for the same contract. Submitting a review automatically updates the freelancer's average rating in the `freelancer_profile` table.

Method	URL	Auth	Role	Tables	Description
GET	/reviews/my	Yes	Any	reviews, contracts, projects	users, bids, Returns all reviews received by the logged in user ordered by date. Includes reviewer name and project title.

Method	URL	Auth	Role	Tables	Description
GET	/reviews/given	Yes	Any	reviews, contracts, projects	users, bids, Returns all reviews written by the logged in user. Includes reviewee name and project title.
POST	/reviews	Yes	Any	reviews, contracts, freelancer_profile, notifications	Submits a review. Validates contract is completed and no duplicate review exists. Updates the freelancer average rating with AVG(rating). Notifies the reviewee.

Table 7.6: Reviews Endpoints

7.7.1 Submit Review Request Body

Listing 7.8: POST /reviews Request Body

```

1 {
2   "contract_id": 3,
3   "reviewee_id": 7,
4   "rating": 4,           // integer between 1 and 5
5   "comment": "Great work, delivered on time." //
6   optional

```

7.8 Disputes Endpoints

Either party on a contract can raise a dispute. Only one open dispute is allowed per contract at a time. The resolve endpoint has no role restriction at the route level but is intended for admin use in practice.

Method	URL	Auth	Role	Tables	Description
GET	/disputes/my	Yes	Any	disputes, users, contracts, bids, projects	Returns all disputes where the logged in user is either the client or the freelancer on the related contract. Uses OR in the WHERE clause.
POST	/disputes	Yes	Any	disputes, contracts, bids, projects	Raises a new dispute. Validates the user is part of the contract and no open dispute already exists for it. Inserts with status 'open'.
PUT	/disputes/:id/resolve	Yes	Any	disputes	Updates dispute status to 'resolved'. Single UPDATE on the disputes table.

Table 7.7: Disputes Endpoints

7.8.1 Raise Dispute Request Body

Listing 7.9: POST /disputes Request Body

```

1 {
2   "contract_id": 3,
3   "reason":      "Freelancer did not deliver the agreed
4   features."

```

7.9 Notifications Endpoints

Notifications are created automatically by the system during bid acceptance, payment recording, and review submission. Users cannot create notifications manually. The endpoints only support reading and marking as read.

Method	URL	Auth	Role	Tables	Description
GET	/notifications/my	Yes	Any	notifications	Returns all notifications for the logged in user ordered by date sent descending.
GET	/notifications/unread-count	Yes	Any	notifications	Returns a single count of unread notifications for the logged in user. Used to display the alert badge on the dashboard.
PUT	/notifications/:id/read	Yes	Any	notifications	Marks a single notification as read. The WHERE clause includes user_id from the token to prevent marking another user's notifications.
PUT	/notifications/read-all	Yes	Any	notifications	Marks all notifications for the logged in user as read in a single UPDATE statement.

Table 7.8: Notifications Endpoints

7.10 Admin Endpoints

All admin endpoints require a valid admin JWT token. They are protected by both `verifyToken` and the `adminOnly` middleware function. All operations are read-only SELECT queries. No admin endpoint modifies any data.

Method	URL	Auth	Role	Tables	Description
GET	/admin/stats	Yes	Admin	users, projects, bids, contracts, payments, disputes, client_profile, free-lancer_profile	Returns nine platform-wide counts and total revenue. Queries run in parallel via Promise.all.

Method	URL	Auth	Role	Tables	Description
GET	/admin/users	Yes	Admin	users, client_profile, free-lancer_profile	Returns all users with role determined by CASE WHEN and profile data from both profile tables via LEFT JOIN.
GET	/admin/projects	Yes	Admin	projects, users, client_profile, categories, bids	Returns all projects with client name, company name, category, and live bid count.
GET	/admin/bids	Yes	Admin	bids, projects, free-lancer_profile, users	Returns all bids with project title, budget, freelancer name, and experience level.
GET	/admin/contracts	Yes	Admin	contracts, bids, projects, client_profile, users uc, users uf	Returns all contracts. Joins users twice to get both the client name and freelancer name.
GET	/admin/payments	Yes	Admin	payments, contracts, bids, projects, client_profile, free-lancer_profile, users uc, users uf	Returns all payments. Seven-table join. Dual alias pattern for client and freelancer names.
GET	/admin/reviews	Yes	Admin	reviews, users u1, users u2, contracts, bids, projects	Returns all reviews. Joins users twice as u1 and u2 to get reviewer and reviewee names.
GET	/admin/skills	Yes	Admin	users, free-lancer_profile, user_skills, skills	Returns every skill assigned to every freelancer via the user_skills junction table.
GET	/admin/disputes	Yes	Admin	disputes, users, contracts, bids, projects	Returns all disputes with the name of the user who raised each one and the associated project title.

Method	URL	Auth	Role	Tables	Description
GET	/admin/notifications	Yes	Admin	notifications, users	Returns every notification sent on the platform with the recipient name.

Table 7.9: Admin Endpoints

7.11 HTTP Status Codes Used

The backend uses a consistent set of HTTP status codes across all endpoints.

Status Code	Meaning	When It Is Returned
200	OK	Successful GET or PUT request
201	Created	Successful POST that inserted a new row
400	Bad Request	Missing required fields or invalid input values
401	Unauthorized	No token provided in the Authorization header
403	Forbidden	Token is invalid, expired, or wrong role
404	Not Found	The requested resource does not exist
409	Conflict	Duplicate record such as email already registered or bid already exists
500	Internal Server Error	Oracle query failed or unexpected server error

Table 7.10: HTTP Status Codes Used by the SkillLink Backend

7.12 Complete Endpoint Summary

Method	Endpoint	Auth	Role	Description
POST	/api/auth/register	No	Any	Register new user
POST	/api/auth/login	No	Any	Login and get token
POST	/api/admin/auth/login	No	Admin	Admin login
GET	/api/projects	No	Any	Browse all projects
GET	/api/projects/client/:id	Yes	Client	My posted projects
GET	/api/projects/:id	No	Any	Single project detail
POST	/api/projects	Yes	Client	Post new project
DELETE	/api/projects/:id	Yes	Client	Delete a project
GET	/api/bids/project/:id	Yes	Client	Bids on a project
GET	/api/bids/freelancer/:id	Yes	Freelancer	My submitted bids
POST	/api/bids	Yes	Freelancer	Submit a bid
PUT	/api/bids/:id/accept	Yes	Client	Accept a bid
GET	/api/contracts/my	Yes	Client or Freelancer	My contracts

Method	Endpoint	Auth	Role	Description
GET	/api/contracts/:id	Yes	Client or Freelancer	Single contract detail
GET	/api/payments/my	Yes	Client or Freelancer	My payment history
POST	/api/payments	Yes	Client	Record a payment
GET	/api/reviews/my	Yes	Any	Reviews received
GET	/api/reviews/given	Yes	Any	Reviews written
POST	/api/reviews	Yes	Any	Submit a review
GET	/api/disputes/my	Yes	Any	My disputes
POST	/api/disputes	Yes	Any	Raise a dispute
PUT	/api/disputes/:id/resolve	Yes	Any	Resolve a dispute
GET	/api/notifications/my	Yes	Any	My notifications
GET	/api/notifications/unread-count	Yes	Any	Unread count
PUT	/api/notifications/:id/read	Yes	Any	Mark one as read
PUT	/api/notifications/read-all	Yes	Any	Mark all as read
GET	/api/admin/stats	Yes	Admin	Platform statistics
GET	/api/admin/users	Yes	Admin	All users
GET	/api/admin/projects	Yes	Admin	All projects
GET	/api/admin/bids	Yes	Admin	All bids
GET	/api/admin/contracts	Yes	Admin	All contracts
GET	/api/admin/payments	Yes	Admin	All payments
GET	/api/admin/reviews	Yes	Admin	All reviews
GET	/api/admin/skills	Yes	Admin	All freelancer skills
GET	/api/admin/disputes	Yes	Admin	All disputes
GET	/api/admin/notifications	Yes	Admin	All notifications

Table 7.11: Complete SkillLink API Endpoint Reference